

CS349D MiniEngine

Milestone 1 Report: Batching and Continuous Batching

Your Name
SUNetID

April 20, 2026

1 Objective

This milestone upgrades the scheduler from naive single-request execution to:

1. batched decode (multiple requests in one forward pass), and
2. continuous batching (iteration-level admission/removal each scheduler step).

The goal is to improve throughput while keeping generation quality/correctness unchanged.

2 Baseline and Design

2.1 Baseline

The baseline scheduler handles one request at a time:

1. prefill one request,
2. decode it until completion,
3. move to the next request.

This underutilizes the GPU because decode runs as many thin matrix-vector operations instead of a larger batched matrix-matrix operation.

2.2 Milestone 1 Design

I implemented two changes:

1. **Batched decode:** stack each running request's last generated token and run one batched forward call per decode iteration.
2. **Continuous batching:** in every scheduler step, admit new requests (up to `max_running`), prefill admitted requests, decode all active requests once, then retire finished requests.

3 Implementation Details

3.1 Files Modified

- `miniengine/scheduler.py`
- `miniengine/engine.py`
- `miniengine/model.py`

3.2 Model/Attention Interface

I threaded an optional `attn_mask` through: `Attention` $\rightarrow$ `TransformerBlock` $\rightarrow$ `TransformerModel` $\rightarrow$ `CausalLM`. When the mask is present, it is passed to `scaled_dot_product_attention`; otherwise the original causal path is used.

3.3 Batched Decode and KV Cache Handling

For a batch with per-request cache lengths $\{L_i\}$:

1. Compute `max_cache_len = max( $L_i$ )`.
2. Pad each request's per-layer KV cache to `max_cache_len`.
3. Build an attention mask over key positions `K = max_cache_len + 1`.
4. Run one batched decode forward pass.
5. For request i , keep old valid KV prefix `[:, :, :  $L_i$ , :]` and append only the newly generated token KV.

3.4 Mask Correctness Note

The decode mask marks valid positions as:

- old real cache tokens `[0,  $L_i$ )`,
- and the newly appended token at `max_cache_len`.

Padding region between L_i and `max_cache_len-1` is masked out. Mask shape is `(B, 1, 1, K)`.

3.5 Other Code Changes (If Any)

If you changed additional parts beyond scheduling, explain them here and justify why they helped correctness/performance.

- Example: tokenizer-side optimization
- Example: stopping criteria cleanup
- Example: stream-path overhead reduction

4 Experimental Setup

4.1 Hardware

- CPU: *fill in*
- RAM: *fill in*
- GPU: *fill in*
- CUDA / driver version: *fill in*

4.2 Software

- Model: Qwen/Qwen3-4B-Instruct-2507 (or your chosen model)
- Dtype: bfloat16 / float16 (*fill in*)
- Command used to launch server:

```
python -m miniengine --model Qwen/Qwen3-4B-Instruct-2507 --dtype bfloat16 --max-running 16
```

- Benchmark command:

```
python -m benchmark.bench_serving --input-len 512 --output-len 256 --concurrencies 1,4,8,16
```

5 Results

5.1 Raw Metrics

Replace placeholders with your measured numbers.

Method @ Concurrency	TTFT p50 (ms)	Completion p50 (ms)	TPOT p50 (ms)	Gen Throughput (tok/s)
Baseline @ 1	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Baseline @ 4	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Baseline @ 8	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Baseline @ 16	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Batched+Continuous @ 1	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Batched+Continuous @ 4	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Batched+Continuous @ 8	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>
Batched+Continuous @ 16	<i>fill</i>	<i>fill</i>	<i>fill</i>	<i>fill</i>

Table 1: Serving metrics comparison between baseline and Milestone 1 implementation.

5.2 Performance Gain Summary

Summarize key gains, e.g.:

- Throughput improvement at concurrency 16: $X\times$
- Completion latency reduction at concurrency 16: $Y\%$
- TTFT impact: *brief note*

6 Discussion

6.1 What Worked

Briefly discuss why batching improved utilization in your implementation.

6.2 Tradeoffs / Limitations

Examples:

- prefill is still unbatched,
- extra overhead from per-step padding,
- sensitivity to request length distribution.

6.3 Hardware Choice Rationale

Since this milestone allows flexible hardware, explain why your CPU/GPU config was chosen and any observed bottlenecks.

7 Conclusion

This milestone demonstrates that batched decode + continuous batching provides measurable performance gain over the naive baseline while preserving generation correctness.

Submission Checklist

- PDF includes design and implementation description.
- PDF includes baseline vs optimized metrics.
- Hardware/software setup is documented.
- Any extra code changes are justified.